WIZM Yellow Paper

State Transition, PoA Consensus, Mempool, Fees, and Network Semantics
Formal Specification of the Current Five-Validator Network

WIZM Protocol Engineering

29 October 2026

Abstract

This document specifies the current WIZM ledger as a deterministic state transition system. WIZM is a five-validator proof-of-authority payment chain with transparent and stealth transfers. It uses Falcon-512 signatures, ML-KEM-512 for committee and mempool key establishment, ML-KEM-768 for inter-validator transport, BLAKE3 hashing, and AES-256-GCM mempool encryption. Stealth transfers provide recipient unlinkability only; sender and amount are public. The document defines the state, transaction validity, execution, encrypted mempool transitions, PoA slot schedule, fallback and fork rules, notary finality, synchronization, snapshots, fees, emission, capacity bounds, and the safety and liveness claims supported by the current design.

Contents

1 Notation and Conventions **2**

2 Parameters **3**

3 Cryptographic Primitives **3**
3.1 Signature scheme . . . 3
3.2 Key encapsulation and encryption . . . 4
3.3 Domains . . . 4

4 State **4**

5 Transactions **4**
5.1 Transparent layout . . . 4
5.2 Stealth layout . . . 5
5.3 Length classification . . . 5

6 Validity Predicates **5**

7 Execution and Fees **6**
7.1 Fee function . . . 6
7.2 Transparent execution . . . 6
7.3 Stealth execution . . . 6

8 Block Execution **7**

9 Mempool Transition System **7**

10 PoA Slot Schedule **8**
10.1 Committee . . . 8
10.2 Slot arithmetic . . . 8
10.3 Suppression gates . . . 9

11 Fork Convergence **9**

12 Finality **9**

13 Synchronization **10**

14 Tokenomics Functions **10**

15 Capacity **10**

16 Safety and Liveness **11**

17 Security Analysis **11**
17.1 Message authentication . . . 11
17.2 Replay and double spend . . . 11
17.3 Resource bounds . . . 11
17.4 Trust boundary . . . 11

18 Conformance Requirements **11**

19 References **12**

1 Notation and Conventions

Definition 1.1 (Basic notation)**.** Let $\{0,1\} = \{0,1\}$. For $n \in \mathbb{N}$, $\{0,1\}^n$ is the set of n-bit strings. $\{0,1\}^{160}$ is the set of 20-byte account addresses and $\{0,1\}^{256}$ is the set of 32-byte hashes. $\|$ denotes concatenation. Integers in wire formats are unsigned little-endian unless stated otherwise. A function $H : \{0,1\}^* \to \{0,1\}^{256}$ denotes BLAKE3-256.

Definition 1.2 (Execution environments)**.** A *protocol instance* is a tuple

$$\mathcal{W} = (N, \tau, T_{\max}, S_{\max}, H_0, \mathcal{C}_0),$$

where N is the committee size, τ is the steady slot duration, $T_{\max}$ is the block transaction cap, $S_{\max}$ is the block byte cap, H_0 is the genesis hash, and $\mathcal{C}_0$ is the genesis committee. The reference deployment has

$$N = 5, \qquad \tau = 10 \text{ s}, \qquad T_{\max} = 1200, \qquad S_{\max} = 8 \cdot 1024^2.$$

Definition 1.3 (Address)**.** For a Falcon-512 public key pk,

$$A(pk) = H(pk)[0:20] \in \{0,1\}^{160}.$$

An address is a 20-byte value; it is not a network identifier.

Definition 1.4 (Token units)**.** One WIZM equals 10^8 base units. The total supply is

$$\mathsf{SUPPLY} = 8{,}312{,}029{,}999{,}000{,}000$$

base units. The base fee is 10,000 base units and the ingress and transfer minimum is 1,000,000 base units.

2 Parameters

Parameter	Value
Chain identifier	7777
Committee size N	5
Quorum q	$\lfloor N/2 \rfloor + 1 = 3$
Steady slot τ	10 s
Genesis slot	240 s
Slow-start slots	60 s for slots 1–9
Slot grace	4 s
Fallback delay D_f	60 s
Rebroadcast delay	60 s
Mesh settle	150 s after boot
Buffered-proposal suppression	120 s
Epoch length L_e	360 heights
Finality recovery lag L_f	128 blocks
Checkpoint interval	100 heights
Max transactions/block $T_{\max}$	1,200
Max body bytes $S_{\max}$	8 MiB
Sealed throughput $\Theta_{\max}$	120 TPS
Mempool byte cap	500 MiB
Mempool entry cap	200,000
Mempool TTL	5,400 s
Mempool RSS ceiling	5.5 GiB
Peer cap	128 connections
Gossip mesh target	4 (high 6)
Base fee f_0	10,000 base units
Burst-free wallet window W	20 txs/block
Burst multiplier m	10
JSON-RPC body cap	5 MiB
gRPC message cap	4 MiB
Public per-IP read budget	500 req/s
Batch cap	100 items
Snapshot interval	2,000 heights
Body retention	4,000 heights
Header retention	200,000 heights
Rewind journal	32 blocks

3 Cryptographic Primitives

3.1 Signature scheme

Definition 3.1 (Falcon-512)**.** $\Pi_{\text{sig}} = (\mathsf{Gen}, \mathsf{Sign}, \mathsf{Verify})$ is Falcon-512. A public key has 897 bytes and a detached signature is a variable-length compressed encoding in $[512, 752]$ bytes. Correctness requires

$$\Pr[\mathsf{Verify}(pk, m, \mathsf{Sign}(sk, m)) = 1] = 1 - \mathsf{negl}.$$

The implementation rejects malformed key and signature lengths before verification.

3.2 Key encapsulation and encryption

Definition 3.2 (ML-KEM and AES-GCM)**.** ML-KEM-512 is used for committee and mempool key establishment. ML-KEM-768 is used for transport key establishment. AES-256-GCM with a fresh nonce and an authentication tag protects mempool payloads. The committee KEM key pair is held by the validator set; RPC clients do not receive it.

3.3 Domains

Consensus signatures are bound to a chain/genesis domain:

$$\mathrm{dom} = H(\texttt{wizm-consensus-domain-v1} \parallel \mathrm{chain_id}_{\mathrm{le64}} \parallel H_0).$$

Votes sign the domain, block hash, and height or round. The domain prevents a signature produced for one chain or fork from being replayed on another.

4 State

Definition 4.1 (Account)**.** An account is a pair

$$\alpha(a) = (\mathrm{balance}(a), \mathrm{nonce}(a))$$

for $a \in \{0,1\}^{160}$, with an optional public-key record. The balance is measured in base units. The nonce is the sequence number of the next value-bearing transaction accepted from a.

Definition 4.2 (Ledger state)**.** A ledger state is

$$\sigma = (\mathcal{A}, \mathcal{K}, \mathcal{M}, \mathcal{T}, \mathcal{C}),$$

where:

- $\mathcal{A} : \{0,1\}^{160} \rightharpoonup \mathbb{N} \times \mathbb{N}$ maps addresses to balance and nonce;
- $\mathcal{K} \subset \{0,1\}^{256}$ is the set of spent stealth key images;
- $\mathcal{M}$ is block and header metadata used for recovery;
- $\mathcal{T}$ is the tokenomics state, including total emitted, burned fees, and distributed fees;
- $\mathcal{C}$ is the validator committee view for the current epoch.

Definition 4.3 (Tokenomics state)**.** The tokenomics state is

$$\mathcal{T} = (E, B, D),$$

where E is total emitted base units, B is total burned fees, and D is total fees distributed to validators. The executor updates $\mathcal{T}$ from the exact fees charged in a block.

5 Transactions

5.1 Transparent layout

Definition 5.1 (Transparent transfer)**.** A transparent transfer is

$$tx = [from : 20] \parallel [to : 20] \parallel [amount : 8] \parallel [nonce : 8] \parallel [pk : 897] \parallel [sig : 512..752].$$

Its body is

$$body = from \parallel to \parallel \mathrm{le64}(amount) \parallel \mathrm{le64}(nonce).$$

The wire size is 1,465–1,705 bytes. The sender, recipient, and amount are public. The signature covers *body*.

5.2 Stealth layout

Definition 5.2 (Stealth transfer)**.** A stealth transfer is

$$tx = [sender : 20] \| [stealth : 20] \| [ephemeral : 32] \| [view_tag : 1] \| [amount : 8] \| [key_image : 32] \| [pk : 897] \| [sig :$$

Its body is the first 113 bytes. The wire size is 1,522–1,762 bytes. The sender and amount are public. The one-time `stealth` address hides the link to the recipient's published meta-address. The key image prevents a second spend.

Definition 5.3 (Stealth derivation)**.** Let pk_s be the recipient spend public key and let e be an ephemeral public value. Define

$$s = H(e), \qquad stealth = H(pk_s \parallel s)[0:20], \qquad view_tag = s[0].$$

The recipient uses the view secret to recognize candidate outputs and recomputes the expected one-time address. The sender remains visible in the transaction, and the amount is an unencrypted 8-byte field.

Definition 5.4 (Key image)**.** For an ephemeral value e and one-time address A_s,

$$key_image = H(e \parallel A_s).$$

The executor rejects a transaction whose key image is already in $\mathcal{K}$; on success it adds the key image.

5.3 Length classification

Transparent and stealth envelopes overlap because Falcon signatures vary in length. Let L be the wire length. The classifier first checks whether L is inside the transparent or stealth envelope. If only one envelope matches, it returns that type. If both match, it parses the public key at offset 56 and tests whether $A(pk)$ equals the first 20 bytes. A match selects transparent; otherwise the transaction is treated as stealth. Signature, address-binding, and key-image verification remain mandatory downstream.

6 Validity Predicates

Definition 6.1 (Signature validity)**.** For a transparent transfer, the signature must verify under pk over $body = from \parallel to \parallel \text{le64}(amount) \parallel \text{le64}(nonce)$. For a stealth transfer, it must verify under pk over the 113-byte body. In both cases, $A(pk)$ must equal the declared sender.

Definition 6.2 (Transfer policy)**.** For a value-bearing transaction:

$$amount \geq 1{,}000{,}000,$$

and the wire length must lie in the standard envelope for its classified type. A transaction outside its envelope or below the minimum amount is rejected before admission.

Definition 6.3 (Account validity)**.** For a transparent transfer from a with nonce n and amount v, the state must satisfy:

$$\mathcal{A}(a) = (\beta, \nu), \qquad \nu = n, \qquad \beta \geq v + f,$$

where f is the fee computed by the fee function. For a stealth transfer the same balance and key-image checks apply; the sender's nonce is not part of the 113-byte stealth body, and the key image is the replay guard for the one-time output.

7 Execution and Fees

7.1 Fee function

Let i be the 1-based index of a fee-paying transaction from wallet w in a block. The burst multiplier is

$$\mu(i) = \begin{cases} 1, & i \le 20, \\ 10, & i = 21, \\ 100, & i = 22, \\ 1000, & i = 23, \\ \dots \end{cases}$$

Equivalently,

$$\mu(i) = 10^{\max(0, i-20)},$$

with saturation at the maximum representable integer. The charged fee is

$$f(i) = 10{,}000 \cdot \mu(i).$$

7.2 Transparent execution

For a transparent transfer with body $(from, to, v, n)$, fee f, and state σ:

1. require $\mathcal{A}(from) = (\beta, \nu)$ with $\nu = n$ and $\beta \ge v + f$;
2. set $\beta \leftarrow \beta - v - f$ and $\nu \leftarrow \nu + 1$;
3. credit v to to, creating the account if absent;
4. update the state root and tokenomics counters.

7.3 Stealth execution

For a stealth transfer with body $(sender, stealth, ephemeral, view_tag, v, key_image)$:

1. require $key_image \notin \mathcal{K}$;
2. require the sender balance $\beta \ge v + f$;
3. debit $v + f$ from the sender;
4. credit v to the one-time address;
5. insert key_image into $\mathcal{K}$;
6. update the state root and tokenomics counters.

Proposition 7.1 (Conservation per transfer)**.** *If execution succeeds, the sum of debits and credits in a transfer equals the fee: the sender loses $v + f$, the recipient gains v, and f is recorded in the block's fee accumulator.*

Proposition 7.2 (Nonce monotonicity)**.** *For a successful transparent transaction from a with nonce ν, the next accepted transparent transaction from a must use nonce $\nu + 1$.*

8 Block Execution

Definition 8.1 (Block)**.** A block is a tuple

$$B = (h, p, \text{producer}, \text{timestamp}, txs, \text{tx_root}, \text{state_root}, \text{sig}),$$

where h is the height, p is the parent hash, txs is a sequence of raw transactions, and sig is the producer's Falcon-512 signature over the header hash and height.

Definition 8.2 (Block validity)**.** A block B is valid relative to state σ and committee C when:

1. $h = \text{tip} + 1$ and p equals the current tip hash;
2. the producer is $C[(h + a) \bmod |C|]$ for some attempt a that is due;
3. the producer signature verifies under the committee key;
4. $|txs| \leq 1{,}200$ and the encoded body is at most 8 MiB;
5. tx_root is the root over the transaction bytes;
6. each transaction passes classification, signature, policy, balance, and replay checks;
7. the resulting state root equals the persisted root after execution.

Definition 8.3 (Fee accounting)**.** For each block, let F be the sum of fees of transactions that actually applied. If the block height is inside the emission window,

$$B \leftarrow B + F, \qquad D \leftarrow D,$$

otherwise the fee is divided among N validators, the divisible part is added to D, and the remainder is added to B.

Definition 8.4 (Block reward)**.** For emission period j,

$$R_j = \frac{E_j}{B_{\text{period}} N},$$

where $B_{\text{period}} = 6{,}311{,}520$ blocks. The reward is credited to each validator's configured address.

9 Mempool Transition System

Definition 9.1 (Entry)**.** A mempool entry is a tuple

$$e = (\text{id}, \text{ciphertext}, \text{author}, \text{nonce}, \text{arrival}, \text{size}),$$

where id is the raw transaction hash, ciphertext is the encrypted transaction body, and author identifies the submitting lane. Entries whose layout exposes a nonce carry it in the header.

Definition 9.2 (Admission)**.** An entry is admitted when:

- the pool byte, entry, and RSS limits permit it;
- the transaction classification and transfer policy pass;
- the fee payer's live balance is at least the ingress minimum;
- the transaction signature and address binding pass;
- a stealth key image is not already spent or pending;

- the entry is not a duplicate raw transaction.

Definition 9.3 (Floor and parking)**.** For each sender a, let $\phi(a)$ be the next nonce floor. A transparent entry with nonce n is immediately drainable when $n = \phi(a)$. If $n > \phi(a)$, the entry is parked until the gap closes. If the gap cannot close before the bounded hold timeout, the parked entries are released or evicted according to the pool policy.

Definition 9.4 (Drain)**.** A producer drains entries in a deterministic order subject to $T_{\max}$ and $S_{\max}$. The drain respects the per-sender floor and never emits a nonce ahead of its predecessor. Entries that cannot be applied are excluded from the economic fee accumulator.

Definition 9.5 (Eviction)**.** Entries are evicted when their TTL expires, when the pool is above its byte or entry ceiling, or when the process RSS gate requires shedding. The pool keeps counters for expired, tail-evicted, duplicate, and forfeited entries so that eviction is observable.

10 PoA Slot Schedule

10.1 Committee

Definition 10.1 (Committee)**.** A committee is an ordered list of seats

$$C = (c_0, \ldots, c_{N-1}), \qquad c_i = (\text{index}_i, \text{pubkey}_i),$$

where each active seat has a Falcon-512 public key. Seats are stable; a removed seat is a hole and does not shift later indices.

Definition 10.2 (Quorum)**.** For n active seats,

$$q = \left\lfloor \frac{n}{2} \right\rfloor + 1.$$

For the reference five-seat committee, $q = 3$.

10.2 Slot arithmetic

Definition 10.3 (Producer)**.** At height h and attempt a,

$$\text{producer}(h, a) = C[(h + a) \bmod N].$$

Attempt 0 is the normal round-robin producer. Attempt $a > 0$ is a fallback producer after a expired slots for the same height.

Definition 10.4 (Slow start)**.** Slot 0 has duration 240 seconds. Slots 1 through 9 have duration 60 seconds. Slot 10 and later have duration $\tau = 10$ seconds. A slot has a four-second grace window after its nominal end.

Definition 10.5 (Attempt due)**.** Let $S(t)$ be the current slot at wall time t and let

$$\text{due}(h, t) = \max\left(0, S(t) - S(\text{start}(h))\right).$$

A block for height h produced at attempt $a \leq \text{due}(h, t)$ is eligible; later attempts are not yet due. The fallback spacing is 60 seconds.

10.3 Suppression gates

A node suppresses its own proposal when:

- it booted less than 150 seconds ago (mesh settle);
- it holds a fresh competing proposal for the same height (bounded by 120 seconds);
- a validated mesh sample is ahead and recent;
- it is executing a catch-up or sync pipeline;
- its previous seal is within the rebroadcast window.

These gates protect a half-formed mesh from permanent same-height forks. They do not alter the producer schedule or the finality quorum.

11 Fork Convergence

Definition 11.1 (Better tip)**.** For two valid versions of the same height, the better version has:

1. the smaller production attempt; or
2. for equal attempts, the lexicographically smaller block hash.

Proposition 11.1 (Strict improvement)**.** *The better-tip relation is a strict total order on distinct valid versions of the same height. Repeated adoption therefore cannot oscillate between two versions.*

Definition 11.2 (Unwind)**.** When a node adopts a better same-height version, it unwinds its local tip by at most one height, re-executes the winning block, and updates its machine, storage, and mempool floors. The unwind is bounded by the persisted rewind journal.

12 Finality

Definition 12.1 (Notarization vote)**.** A notarization vote is

$$v = (\text{height}, \text{block_hash}, \text{signer}, \text{domain}, \text{signature}),$$

where the signature covers

$$\texttt{wizm/notary/v2} \parallel \text{domain} \parallel \text{block_hash} \parallel \text{le64}(\text{height}).$$

The wire vote uses the corresponding `wizm/vote/v2` envelope; the node verifies the wire signature against the committee key and records the verified vote.

Definition 12.2 (Finalized height)**.** Let $\mathcal{V}(h)$ be the set of valid signers with votes for height h on one block hash. Height h is finalized when

$$|\mathcal{V}(h)| \geq q$$

and every height below h is already finalized. The finalized watermark is the greatest such contiguous height.

Proposition 12.1 (Contiguity)**.** *A quorum at height h does not advance the watermark while a lower height lacks a quorum. This prevents a gap from being silently treated as final.*

Definition 12.3 (Recovery floor)**.** A node that restores state or adopts a validated sync prefix may set its recovery watermark to

$$F = \max(0, \text{tip} - 128).$$

The floor is a recovery aid over locally validated history; it is not a claim that the historical heights were individually re-voted in the current process.

13 Synchronization

Definition 13.1 (Sync request)**.** A node requests a range

$$[\text{from}, \text{from} + \ell)$$

from a peer, where ℓ is bounded by the sync window. The response contains full block bodies. The receiver validates parent links, producer signatures, transaction roots, and execution before adopting the range.

Proposition 13.1 (No unvalidated adoption)**.** *A node may update its mesh-best sample from a peer height only after it has validated a linking block or prefix. A far-future height claim alone cannot raise the local tip or suppress local production indefinitely.*

Definition 13.2 (Snapshot)**.** A snapshot is a tuple

$$\Sigma = (h, \text{root}, \text{id}, \text{parts}),$$

where h is the snapshot height, root is the state root, id is a content identifier, and parts is the set of uploaded slices. Import verifies the part count, content identifier, state root, and height before sealing the snapshot.

14 Tokenomics Functions

Definition 14.1 (Emission)**.** For height h, the emission period is

$$j(h) = \min\left(4, \left\lfloor \frac{h}{B_{\text{period}}} \right\rfloor\right).$$

The per-validator block reward is

$$R(h) = \frac{E_{j(h)}}{B_{\text{period}} N}$$

while $h < 5B_{\text{period}}$; otherwise $R(h) = 0$.

Definition 14.2 (Fee phase)**.** For $h < 5B_{\text{period}}$, fees are burned. After that, fees are divided among validators and any indivisible remainder is burned.

Proposition 14.1 (Fee non-negativity)**.** *The fee accumulator is monotone non-decreasing and saturates rather than wrapping. A block cannot reduce a previously persisted burned-fee counter.*

15 Capacity

Proposition 15.1 (Sealed throughput bound)**.** *For any block sequence with slot duration at least ten seconds and at most 1,200 transactions per block, the sealed transaction rate is at most*

$$\Theta_{\max} = \frac{1{,}200}{10} = 120 \text{ TPS}.$$

Proposition 15.2 (Wallet lower bound)**.** *To fill a 1,200-transaction block without a burst multiplier, a generator needs at least*

$$\left\lceil \frac{1{,}200}{20} \right\rceil = 60$$

distinct fee-paying wallets, assuming each wallet contributes at most 20 fee-paying transactions to the block.

16 Safety and Liveness

Definition 16.1 (Crash-fault model)**.** The reference network has $N = 5$ seats and quorum $q = 3$. A crashed or unavailable seat stops producing and voting. The model does not make claims about arbitrary equivocation by a validator with a valid key.

Proposition 16.1 (Liveness with three seats)**.** *If at least three seats are online and follow the schedule, the chain can produce and finalize. With two online seats, the quorum of three cannot be reached and finality halts until a third seat returns.*

Proposition 16.2 (Finality monotonicity)**.** *The finalized watermark is monotone under the protocol. A rewind operation is an explicit operator action, not a routine consensus transition.*

17 Security Analysis

17.1 Message authentication

All consensus messages are signed and domain-bound. A vote or seal from one chain/genesis domain does not verify on another. Transaction signatures bind the sender address to the embedded public key; the address is checked before execution.

17.2 Replay and double spend

Transparent transactions use sequential nonces. Stealth transfers use key images. The mempool rejects duplicate transaction hashes and spent or pending key images. A sealed transaction is also checked against the committed hash set.

17.3 Resource bounds

The protocol fixes block count and bytes, mempool bytes and entries, RPC bodies and batches, peers, and per-wallet fees. These are not optional implementation details: a conforming node must enforce them before expensive work. Public RPC gateways additionally apply per-IP and per-key limits.

17.4 Trust boundary

Validator keys are not available to RPC clients. Public API keys can submit transactions but cannot sign blocks or votes. The operator rewind path is separate from normal submission. Wallet signing keys stay in wallet storage or a loopback sidecar.

18 Conformance Requirements

A conforming implementation must:

- classify transparent and stealth layouts by wire envelope and address binding;
- reject transactions outside the standard size envelope, below the minimum amount, with an invalid signature, with a bad nonce, or with a duplicate key image;
- enforce the 1,200 transaction and 8 MiB block caps;
- enforce the per-wallet burst multiplier;
- verify the scheduled producer and block signature;
- apply the better-tip rule for same-height versions;

- require a quorum of validated votes for finality;
- validate sync prefixes before adoption;
- persist state, fee counters, key images, and block metadata atomically;
- enforce the configured RPC permissions and limits.

19 References

References

[1] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*. 2020.

[2] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé. *CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM*. IEEE EuroS&P, 2018.

[3] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. *BLAKE3: One Function, Fast Everywhere*. 2020.

[4] Protocol Labs. *libp2p: A Modular Peer-to-Peer Networking Stack*. 2024.

[5] Facebook Engineering. *RocksDB: A Persistent Key-Value Store for Flash and RAM Storage*. 2024.

[6] S. De Angelis, L. Aniello, R. Baldoni, F. Lombardi, A. Margheri, and V. Sassone. *PBFT vs Proof-of-Authority: Applying the CAP Theorem to Permissioned Blockchain*. 2018.

[7] N. van Saberhagen. *CryptoNote v2.0*. 2013.